

C2. CI design + Flake & Environment Strategy

1. CI Pipeline Design

1.1 PR Quality Gate

Pull Request → Static / Fast Checks → Contract / Schema Validation → Bake Bronze Docker Image → Dev Segment Deployment → Smoke + Critical API Regression → PR Quality Gate → Bake Gold Docker Image → Test Environment Deployment

Need fast feedback and high signal:

- Static checks: linting, configuration validation, static code analysis.
- Unit tests
- Contract / Schema validation.
- Smoke tests covering critical service availability and core flows.
- Critical API regression tests.
- Negative tests only where they cover high-risk validation behavior.
- Security checks

1.2 Nightly Runs

Assumption is the latest gold docker image is deployed on the test environment.

Scheduled Nightly → Full Regression → Test Environment Confidence

Broader coverage:

- Full API regression.
- All negative and boundary scenarios.
- Flows that use multiple API resources together.
- Idempotency scenarios.
- Concurrency tests.
- Extended end-to-end journeys.

1.3 Pre Release

Pre Release → Full Confidence Suite → Production Deployment

Need maximum confidence before release:

- Full contract validation.
- Full critical regression.
- Critical end-to-end journeys.
- Transaction correctness tests.
- Concurrency and idempotency gates.
- Performance/reliability gates (if required).

2. Flake & Environment Strategy

2.1 Deterministic execution against shared state

- Create isolated test accounts.
- Tests should create/clean-up their own data via fixture or test hooks.
- Tests should not depend upon execution order.
- Tests must be safe to run in parallel.
- Concurrency tests should share state only when sharing that state is the behavior under test.

2.2 Distinguishing a genuine intermittent defect from test flake

Observation	Investigation Direction
Same test and the same input repeatedly reproduce the failure.	Investigate it as a likely product or test issue.
Failure occurs only under parallel execution.	Check for race conditions, locking issues, shared-state problems, and idempotency issues.
Failure follows a particular environment.	Check for environment issues or test-isolation problems.
Failure disappears when test data is isolated.	Treat it as a test-data or shared-state problem, not a product issue.
Failure correlates with external dependency instability.	Check if the environment or its dependencies are causing the issue.

2.3 Running 200 test in parallel

- Create dedicated test accounts/resources where possible to isolate tests.
- Use unique idempotency keys and test data for independent operations.
- Share payment state only when shared state is the behavior being tested.
- Run with pytest-xdist after proving the tests are isolated and parallel-safe.